STAT 302 Fall 2025 - Final Project

Zayna Badr

12/5/2025

The assignment should be completed in an R Markdown document. Submit the assignment electronically on Gradescope by December 12, 2025, at 11:59 PM as a PDF file. No late submission will accepted.

Each answer has to be based on `R` code that shows how the result was obtained. `R` code has to answer the question or solve the task.

NOTE: Keep the following guidelines in mind::

- Choose the most efficient code whenever possible.
- Do not create functions unless needed.
- Keep your code as short as possible.
- Use only base 'R', unless the instructions explicitly request you to use a specific library.
- Use chunk options to suppress unnecessary output or messages in your code.
- Ensure your code fits within the page and does not extend beyond the margins in the PDF.
- Check the grading rubric for all the items which will be graded in this Assignment!

The total number of points of this assignment is 50 points.

Good luck!

K means clustering

In this project, you will implement an unsupervised learning technique called **k-means clustering**. This method is used to partition a dataset into k distinct clusters based on the similarity of the data points. The goal of k-means is to minimize the variance within each cluster, so that points within the same cluster are as similar as possible.

While there is a built-in function in base R that can perform k-means clustering, you are not allowed to use it in this project. Instead, you will manually implement the algorithm from scratch.

The k-means algorithm works as follows:

- Choose k initial centroids (typically randomly selected points from the dataset).
- Assign each data point to the closest centroid, forming KK clusters.
- Recompute the centroids of the clusters by calculating the mean of all points assigned to each cluster.
- Repeat the assignment and update steps until the centroids no longer change (or change very little), indicating convergence.

Your task is to replicate this process and implement the algorithm manually using base `R` functions, without relying on any built-in `R` functions (such as the `knn()` function) and without relying on any specialized clustering libraries.

Consider one of the following data set:

```
taylor_album_songs <- read.csv('https://raw.githubusercontent.com/rfordatascience/tidytuesday/master/da
songs_reduced <- taylor_album_songs[,c("track_number", "energy", "valence")]
songs_reduced_final <- songs_reduced[rowSums(is.na(songs_reduced))==0,]
```

Or use the data set `Mall_Customers.csv` uploaded on Canvas in the folder named `final project`.

```
mall_customers <- read.csv("/Users/zaynabadr/STAT 302/Mall_Customers.csv")
```

In order to simplify your task, if you are working with the `mall_customers` data set, consider only the quantitative variables. If you are working with the Taylor Swift songs, you will work on with energy and valence since track_number is a label rather than a meaningful variable.

```
# Loading dataframe
mall_df <- read.csv("/Users/zaynabadr/STAT 302/Mall_Customers.csv")

# Keeping only quantitative variables (numeric columns)
mall_df$Gender <- NULL
mall_df$CustomerID <- NULL

# Renaming columns for clarity
colnames(mall_df)[2] = "Annual_Income_k"
colnames(mall_df)[3] = "Spending_Score_1_100"

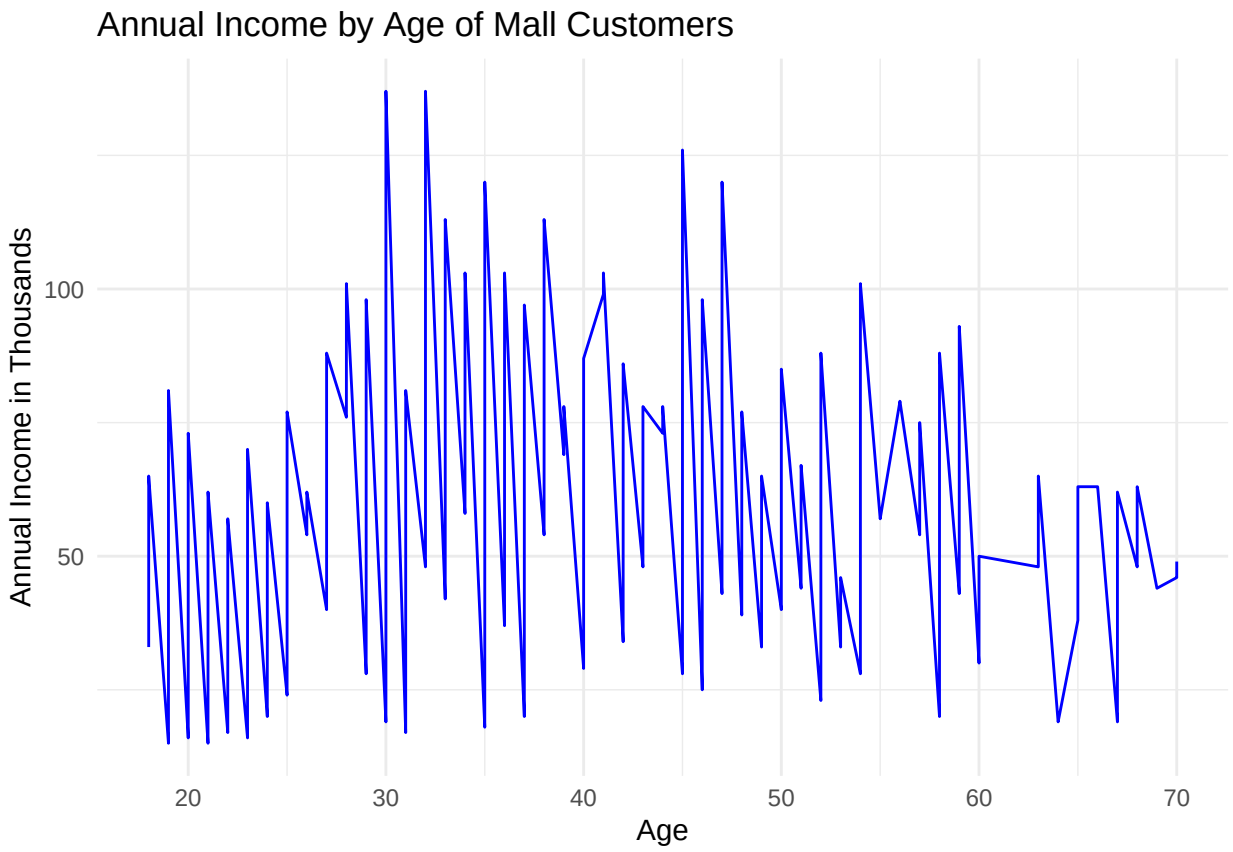
head(mall_df)
```

```
##   Age Annual_Income_k Spending_Score_1_100
## 1  19              15                   39
## 2  21              15                   81
## 3  20              16                    6
## 4  23              16                   77
## 5  31              17                   40
## 6  22              17                   76
```

1. In order to start exploring the dataset, use base `R` or `ggplot` to create data visualizations (at least 2).

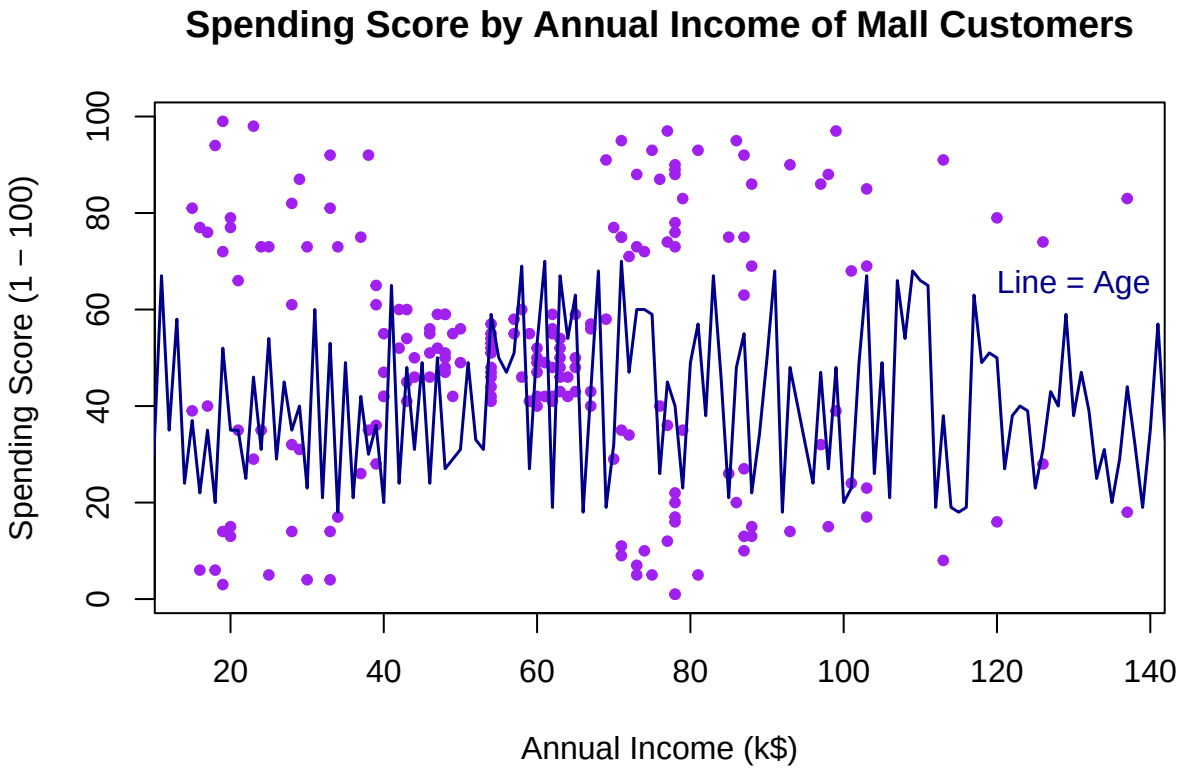
```r
library(ggplot2)

ggplot(data = mall_df, aes(x = Age, y = Annual_Income_k)) +
  geom_line(color = "blue") +
  labs(title = "Annual Income by Age of Mall Customers", x = "Age", y = "Annual Income in Thousands")+
  theme_minimal()
```

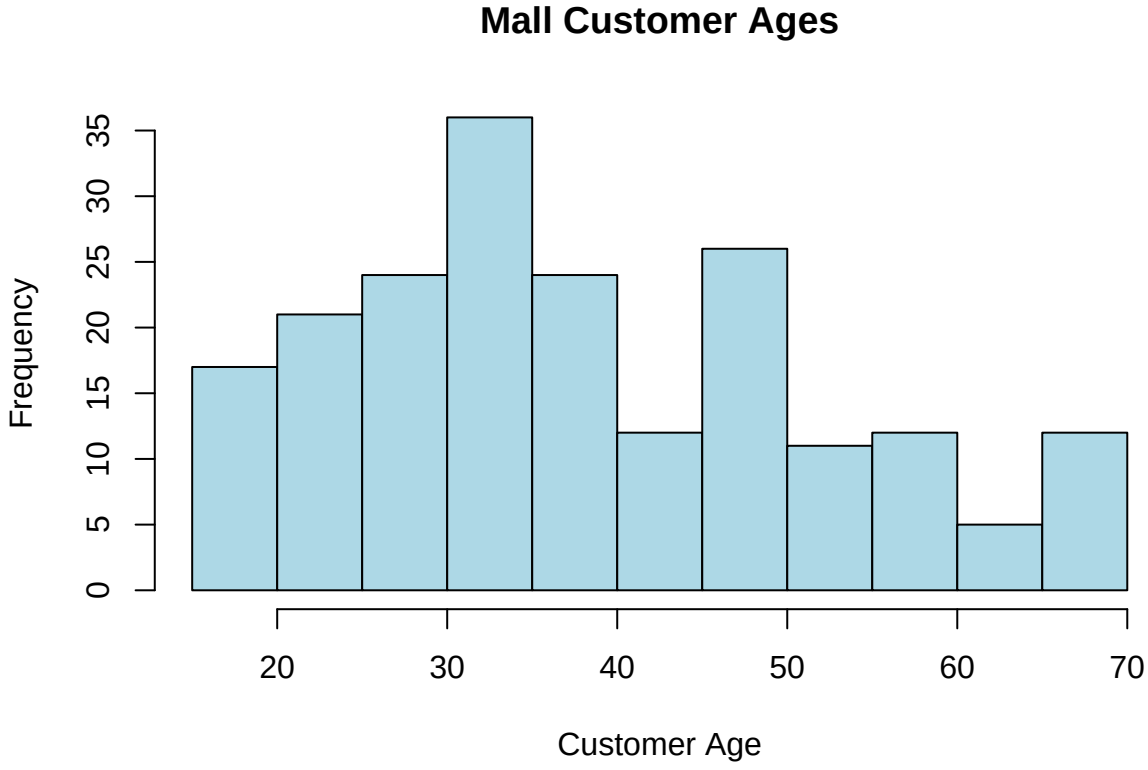



```r
plot(mall_df$Annual_Income_k, mall_df$Spending_Score_1_100, type='p', col='purple', pch = 20,
     xlab="Annual Income (k$)", ylab= "Spending Score (1 - 100)",
     main="Spending Score by Annual Income of Mall Customers")

lines(mall_df$Age, lwd = 1.5, col = 'darkblue')
text(x = 130, y = 65, label = "Line = Age", col = "darkblue") # Label the abline
```




```r
hist(mall_df$Age, col = "lightblue", breaks = 15,
     xlab = "Customer Age", ylab = "Frequency", main = "Mall Customer Ages")
```



2. Randomly choose any three points from the dataset which will be your initial centroids. These will define the centers of the first initial three clusters. This means that you are defining $k = 3$ clusters.

```r
set.seed(0915)
k <- 3
n <- nrow(mall_df)

# Picking random customers to be my initial indices
initial_inds <- sample(1:nrow(mall_df), k)

# Filtering mall_df for these initial indices and converting to matrix for ease
icentroids <- as.matrix(mall_df[initial_inds,])
icentroids
```

```
##     Age Annual_Income_k Spending_Score_1_100
## 52   33              42                   60
## 139  19              74                   10
## 26   29              28                   82
```

3. Measure the distance between all the points in the dataset and each of the three initial clusters. Assign the points to the nearest cluster. Do this in the most efficient way.

```r
distances <- matrix(0, n, k)

# For each point, calculating distance from each centroid
for (i in 1:n) {
  point <- as.numeric(mall_df[i, ])
  distances[i, 1] <- sqrt(sum((point - icentroids[1,])^2))
  distances[i, 2] <- sqrt(sum((point - icentroids[2,])^2))
  distances[i, 3] <- sqrt(sum((point - icentroids[3,])^2))
}

# Assigning cluster by picking the column with the smallest distance from a centroid for each point
assign_cluster <- apply(distances, 1, which.min)
assign_cluster
```

```
##   [1] 1 3 2 3 1 3 2 3 1 3 1 3 1 3 1 3 1 3 1 3 1 3 2 3 1 3 1 1 1 3 2 3 2 3 1 3 1
##  [38] 3 1 3 1 3 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
##  [75] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [112] 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1
## [149] 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 2
## [186] 1 2 1 2 1 2 1 2 1 2 1 2 2 2 2
```

4. After assigning all points to the closest centroid, recalculate the centroids. This is achieved by computing the mean of all the variables of the points assigned to each cluster. This will give you the new centroids.

```r
# Converting df to matrix for ease
mall_m <- as.matrix(mall_df)

# Storing new centroids
new_centroids <- matrix(0, k, ncol(mall_m))

# Calculating new centroids as the mean of all the points assigned to the old centroids
for (i in 1:k) {
  new_centroids[i, ] <- colMeans(mall_m[assign_cluster == i, ])
}

new_centroids
```

```
##          [,1]     [,2]     [,3]
## [1,] 40.45926 58.68148 55.77778
## [2,] 40.24444 82.02222 19.77778
## [3,] 24.85000 24.95000 81.00000
```

5. Repeat the process of assigning points to clusters and updating the centroids until the centroids no longer change.

```
centroids <- new_centroids

repeat {

  # Compute distances to each centroid
  distances <- matrix(0, n, k)
  for (i in 1:n) {
    point <- mall_m[i, ]
    distances[i, 1] <- sqrt(sum((point - centroids[1,])^2))
    distances[i, 2] <- sqrt(sum((point - centroids[2,])^2))
    distances[i, 3] <- sqrt(sum((point - centroids[3,])^2))
  }

  # Assign clusters
  assign_cluster <- apply(distances, 1, which.min)

  # Compute new centroids
  new_centroids <- matrix(0, k, ncol(mall_m))
  for (i in 1:k) {
    new_centroids[i, ] <- colMeans(mall_m[assign_cluster == i, ])
  }

  # Stop when centroids do not change
  if (sqrt(sum((centroids - new_centroids)^2)) == 0) break

  # otherwise continue
  centroids <- new_centroids
}

# These should be equal
new_centroids
```

```
##          [,1]     [,2]     [,3]
## [1,] 32.69231 86.53846 82.12821
## [2,] 40.39474 87.00000 18.63158
## [3,] 40.32520 44.15447 49.82927
```

```
centroids
```

```
##          [,1]     [,2]     [,3]
## [1,] 32.69231 86.53846 82.12821
## [2,] 40.39474 87.00000 18.63158
## [3,] 40.32520 44.15447 49.82927
```

6. Calculate the Within-Cluster-Sum of Squared Errors (WSS) for different values of k (including the $k = 3$). The WSS is given by:

$$\text{WSS} = \sum_{k=1}^{K} \sum_{i=1}^{n_k} \left(\mathbf{x}_i^{(k)} - \boldsymbol{\mu}_k\right)^\top \left(\mathbf{x}_i^{(k)} - \boldsymbol{\mu}_k\right)$$

where:

- $\mathbf{x}_i^{(k)} = (x_{i1}^{(k)}, x_{i2}^{(k)}, \ldots, x_{id}^{(k)})$ is the i-th data point in cluster k, where d is the number of dimensions (features).
- $\boldsymbol{\mu}_k = (\mu_{k1}, \mu_{k2}, \ldots, \mu_{kd})$ is the centroid (mean) of cluster k, calculated as the average of the points in the cluster.
- The term $\left(\mathbf{x}_i^{(k)} - \boldsymbol{\mu}_k\right)^\top \left(\mathbf{x}_i^{(k)} - \boldsymbol{\mu}_k\right)$ represents the squared Euclidean distance between the point $\mathbf{x}_i^{(k)}$ and its corresponding centroid $\boldsymbol{\mu}_k$.

```
# Assigning WSS
WSS <- 0

# Looping through each centroid and assigning points to that centroid cluster
for (i in 1:k) {
  cluster_points <- mall_m[assign_cluster == i, ]
  # Looping through each point in cluster
  for (j in 1:nrow(cluster_points)) {
    point <- cluster_points[j, ]
    # Calculating the distance for each point
    dist <- sum((point - centroids[i,])^2)
    # Summing all distances
    WSS <- WSS + dist
  }
}

WSS
```

```
## [1] 143342.8
```

7. Using either base R or ggplot2, plot WSS-versus-k. Choose the k for which the rate of decrease of WSS sharply changes. In the plot, this is visible as an elbow.

```
# Assigning k values I will be plotting
k_plotting <- 1:15

# Where I will store the WSS value for each k
WSS_plotting <- numeric(length(k_plotting))


for (k in k_plotting) {
  # Reassigning initial centroids for each number of clusters
  initial_inds <- sample(1:n, k)
  centroids <- mall_m[initial_inds, ]

  repeat {

    # Compute distances to each centroid
    distances <- matrix(0, n, k)
    for (i in 1:n) {
      point <- mall_m[i, ]
      for (j in 1:k) {
```

```
        if (k == 1) {
          centroid_extract <- centroids
        } else {
          centroid_extract <- centroids[j, ]
        }
        distances[i, j] <- sqrt(sum((point - centroid_extract)^2))
      }
    }

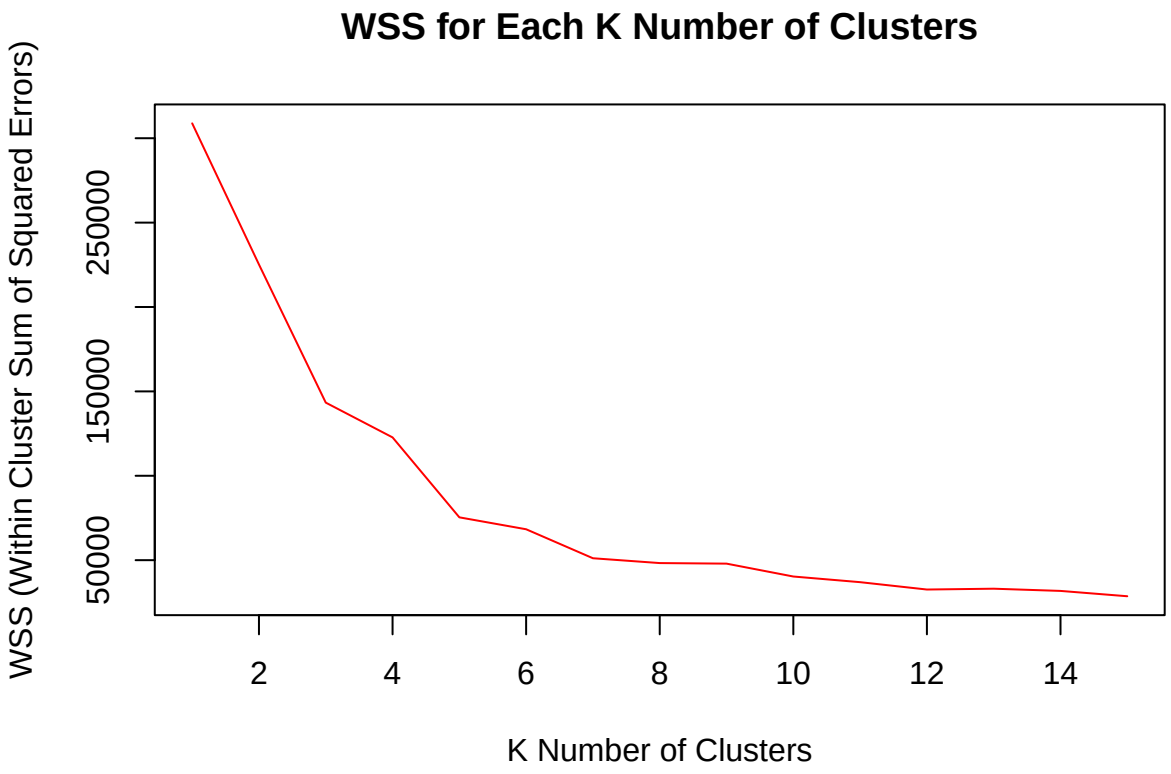
    # Assign clusters
    assign_cluster <- apply(distances, 1, which.min)

    # Compute new centroids
    new_centroids <- matrix(0, k, ncol(mall_m))
    for (i in 1:k) {
      new_centroids[i, ] <- colMeans(mall_m[assign_cluster == i, ])
    }

    # Stop when centroids do not change
    if (sqrt(sum((centroids - new_centroids)^2)) == 0) break

    # otherwise continue
    centroids <- new_centroids
  }
  # Calculate the WSS for each k number of clusters
  WSS <- 0
  for (i in 1:k) {
    cluster_points <- mall_m[assign_cluster == i, ]
    for (j in 1:nrow(cluster_points)) {
      point <- cluster_points[j, ]
      dist <- sum((point - centroids[i,])^2)
      WSS <- WSS + dist
    }
  }
  # Add the WSS for each k number of clusters to the plotting data (I used k - 2 because it was
  WSS_plotting[k] <- WSS
}


# Plotting data
plot(x = k_plotting, y = WSS_plotting, type = "l", col = "red",
     xlab = "K Number of Clusters", ylab = "WSS (Within Cluster Sum of Squared Errors)",
     main = "WSS for Each K Number of Clusters")
```




```r
# Based on the plot the k for which the rate of decrease of WSS flattens out is 5
ideal_k <- 5


# - - -
```

```
    }

    # Assign clusters
    assign_cluster <- apply(distances, 1, which.min)

    # Compute new centroids
    new_centroids <- matrix(0, k, ncol(data))
    for (i in 1:k) {
      cluster_points <- data[assign_cluster == i, ]

      if (nrow(cluster_points) == 0) {
        # Reinitialize an empty cluster to a random data point
        new_centroids[i, ] <- data[sample(1:n_data, 1), ]
      } else {
        new_centroids[i, ] <- colMeans(cluster_points)
      }

    }

    # Stop when centroids do not change
    if (sqrt(sum((centroids - new_centroids)^2)) == 0) break

    # otherwise continue
    centroids <- new_centroids
  }

  # Calculate the WSS for each k number of clusters
  WSS <- 0
  for (i in 1:k) {
    cluster_points <- data[assign_cluster == i, ]
    for (j in 1:nrow(cluster_points)) {
      point <- cluster_points[j, ]
      dist <- sum((point - centroids[i,])^2)
      WSS <- WSS + dist
    }
  }

  return(list(clustering = assign_cluster, WSS = WSS))
}
```

8. Based on this clustering, add a variable called `cluster` to the data set. Use the variables in the data set (both the ones used for clustering and the ones not used for clustering) to visually describe the clusters. Explain what differences you observe.

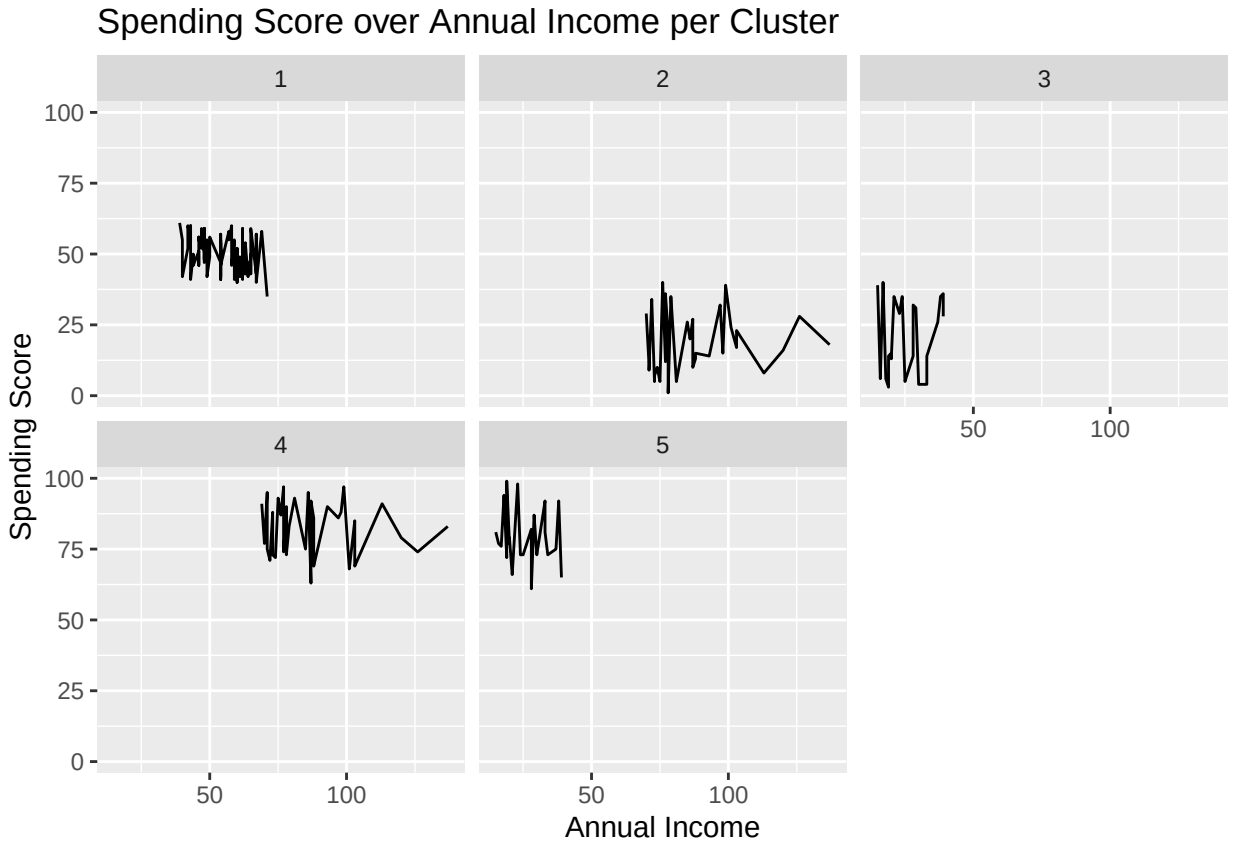
```
# I went back and created ideal_assign_cluster for the value where the curve elbows
ideal_assign_cluster <- kmeans_algorithm(ideal_k, mall_m)[1]
mall_df$cluster <- unlist(ideal_assign_cluster)

# Adding back categorical variables (variables not used for clustering)
mall_df$Gender <- mall_customers$Gender

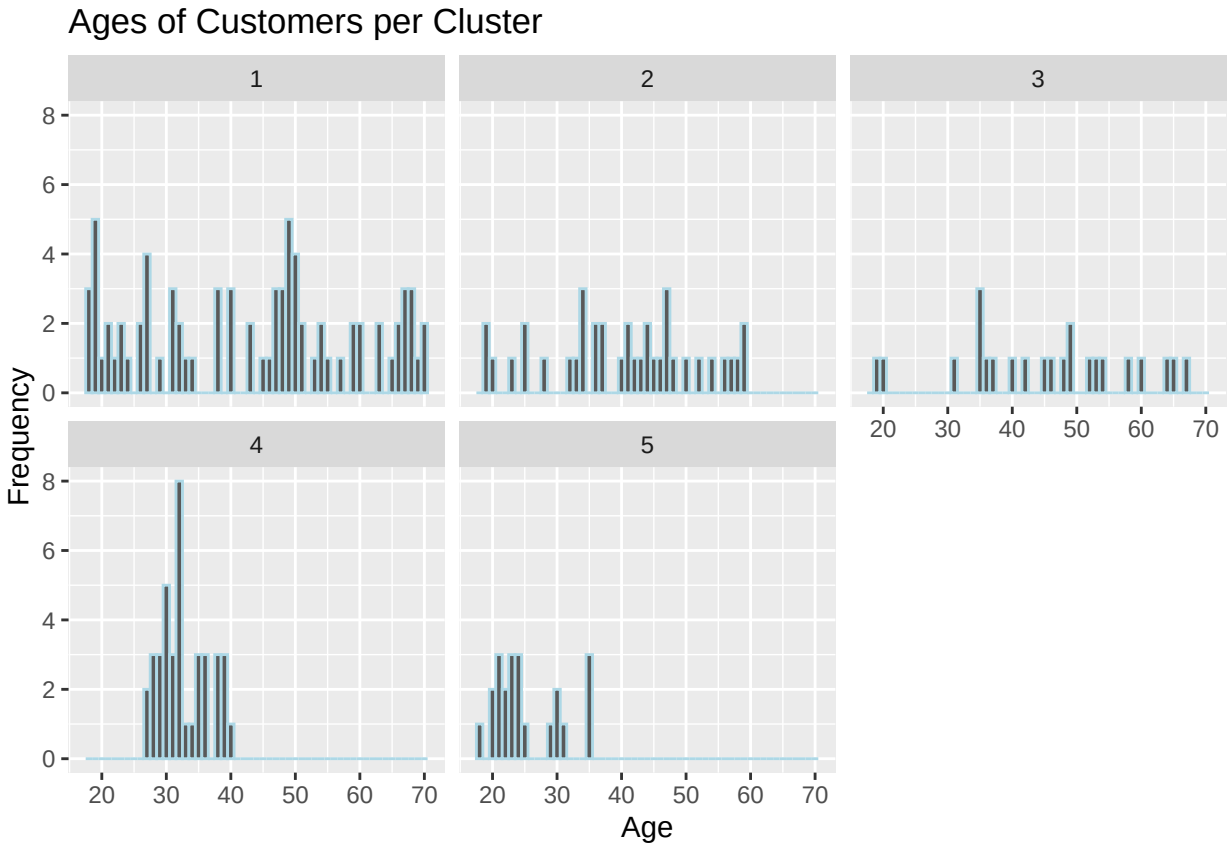
# Plotting:

ggplot(data = mall_df,
```

```
      aes(x = Annual_Income_k, y = Spending_Score_1_100, group = cluster)) +
  geom_line(linewidth = 0.5) +
  facet_wrap(~ cluster) +
  labs(title = "Spending Score over Annual Income per Cluster",
       x = "Annual Income", y = "Spending Score")
```




```
ggplot(data = mall_df, aes(x = Age)) +
  geom_histogram(binwidth = 1, color = 'lightblue') +
  facet_wrap(~ cluster) +
  labs(title = "Ages of Customers per Cluster",
       x = "Age", y = "Frequency")
```



9. K-means clustering performs well in some situations and poorly in others. In general, it works best when data points form well-separated clusters and struggles when clusters are overlapping or absent.

(i) Simulate three datasets representing the following scenarios: * Clearly clustered data * Weakly clustered data * Data with no meaningful clusters Use multivariate normals and simulate two variables for each dataset. Simulate the same number of observations as in the dataset you used for parts 1-8. Use a scatterplot to represent each dataset so that the clustering structure (or lack thereof) is visually clear.

(ii) Apply k-means to each dataset and determine the best value of k for each scenario. How does k-means behave in each scenario?

(iii) Because k-means starts with random initializations, repeat the algorithm using 10 different random seeds. For each pair of data points, compute the proportion of times they end up in the same cluster. How does this stability measure differ across the three scenarios?

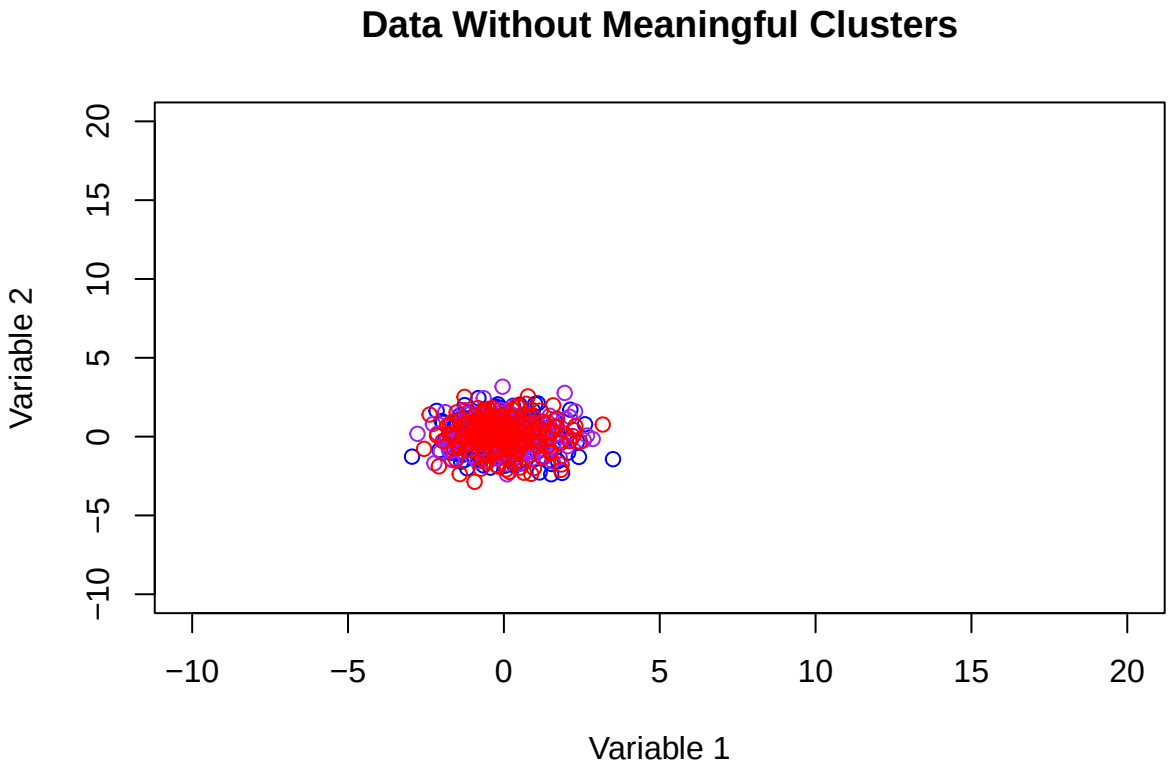
```
# PART ONE: Simulating Datasets

# Data Without Meaningful Clusters:

none_group1 <- MASS::mvrnorm(n = n, mu = c(0,0), Sigma = matrix(c(1, 0, 0, 1), nrow = 2, ncol = 2))
none_group2 <- MASS::mvrnorm(n = n, mu = c(0,0), Sigma = matrix(c(1, 0, 0, 1), nrow = 2, ncol = 2))
none_group3 <- MASS::mvrnorm(n = n, mu = c(0,0), Sigma = matrix(c(1, 0, 0, 1), nrow = 2, ncol = 2))

nocluster_df <- rbind(none_group1, none_group2, none_group3)

plot(none_group1, xlim = c(-10, 20), ylim = c(-10, 20), col = 'blue',
     xlab = "Variable 1", ylab = "Variable 2", main = "Data Without Meaningful Clusters")
points(none_group2, col = 'purple')
points(none_group3, col = 'red')
```

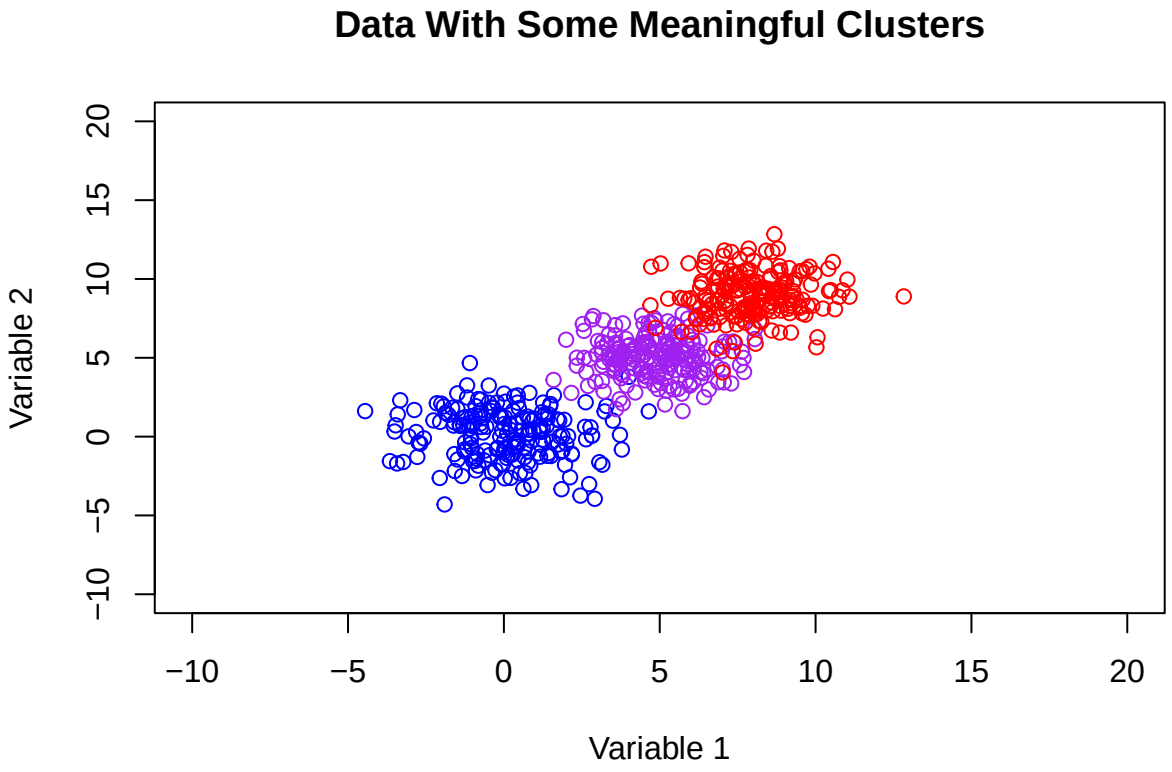



```r
# Data With Some Meaningful Clusters:

some_group1 <- MASS::mvrnorm(n = n, mu = c(0,0), Sigma = matrix(c(3, 0, 0, 3), nrow = 2, ncol = 2))
some_group2 <- MASS::mvrnorm(n = n, mu = c(5,5), Sigma = matrix(c(1.8, 0, 0, 1.8), nrow = 2, ncol = 2))
some_group3 <- MASS::mvrnorm(n = n, mu = c(8,9), Sigma = matrix(c(1.8, 0, 0, 1.8), nrow = 2, ncol = 2))

some_cluster_df <- rbind(some_group1, some_group2, some_group3)

plot(some_group1, xlim = c(-10, 20), ylim = c(-10, 20), col = 'blue',
     xlab = "Variable 1", ylab = "Variable 2", main = "Data With Some Meaningful Clusters")
points(some_group2, col = 'purple')
points(some_group3, col = "red")
```

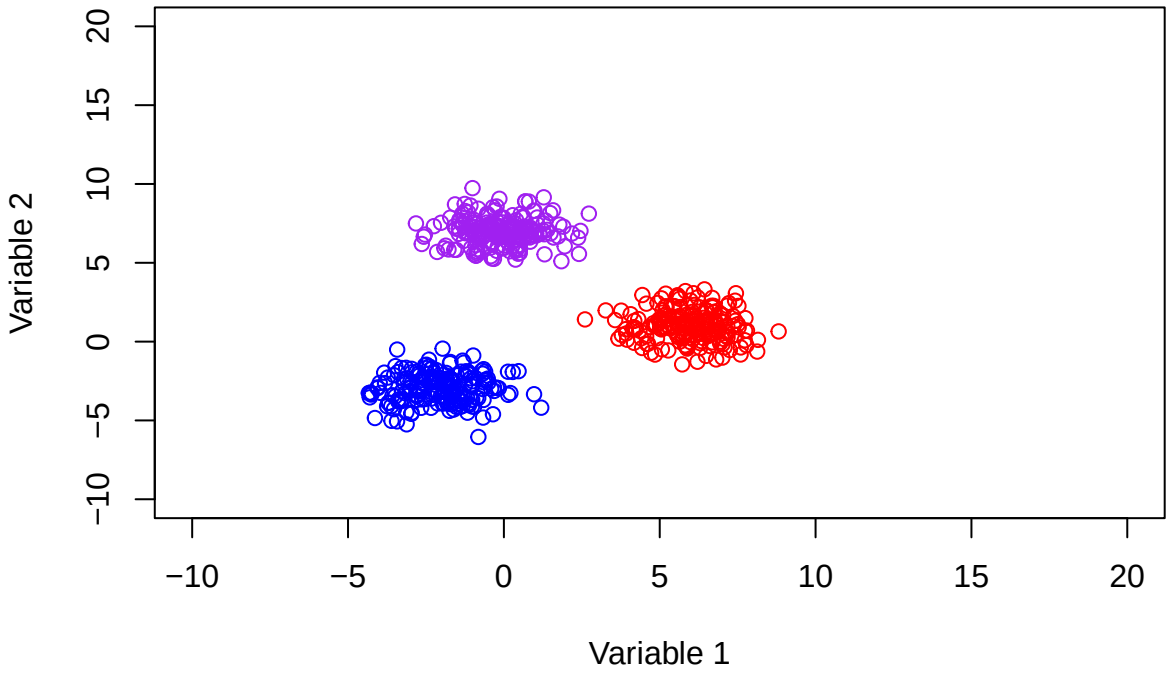



```r
# Data With Clear Clusters:

clear_group1 <- MASS::mvrnorm(n = n, mu = c(-2,-3), Sigma = matrix(c(1, 0, 0, 1), nrow = 2, ncol = 2))
clear_group2 <- MASS::mvrnorm(n = n, mu = c(0,7), Sigma = matrix(c(1, 0, 0, 0.8), nrow = 2, ncol = 2))
clear_group3 <- MASS::mvrnorm(n = n, mu = c(6,1), Sigma = matrix(c(1, 0, 0, 1), nrow = 2, ncol = 2))

clear_cluster_df <- rbind(clear_group1, clear_group2, clear_group3)

plot(clear_group1, xlim = c(-10, 20), ylim = c(-10, 20), col = 'blue',
     xlab = "Variable 1", ylab = "Variable 2", main = "Data With Clear Clusters")
points(clear_group2, col = 'purple')
points(clear_group3, col = "red")
```




```r
# PART TWO: Apply k means and determine the best k for each dataset

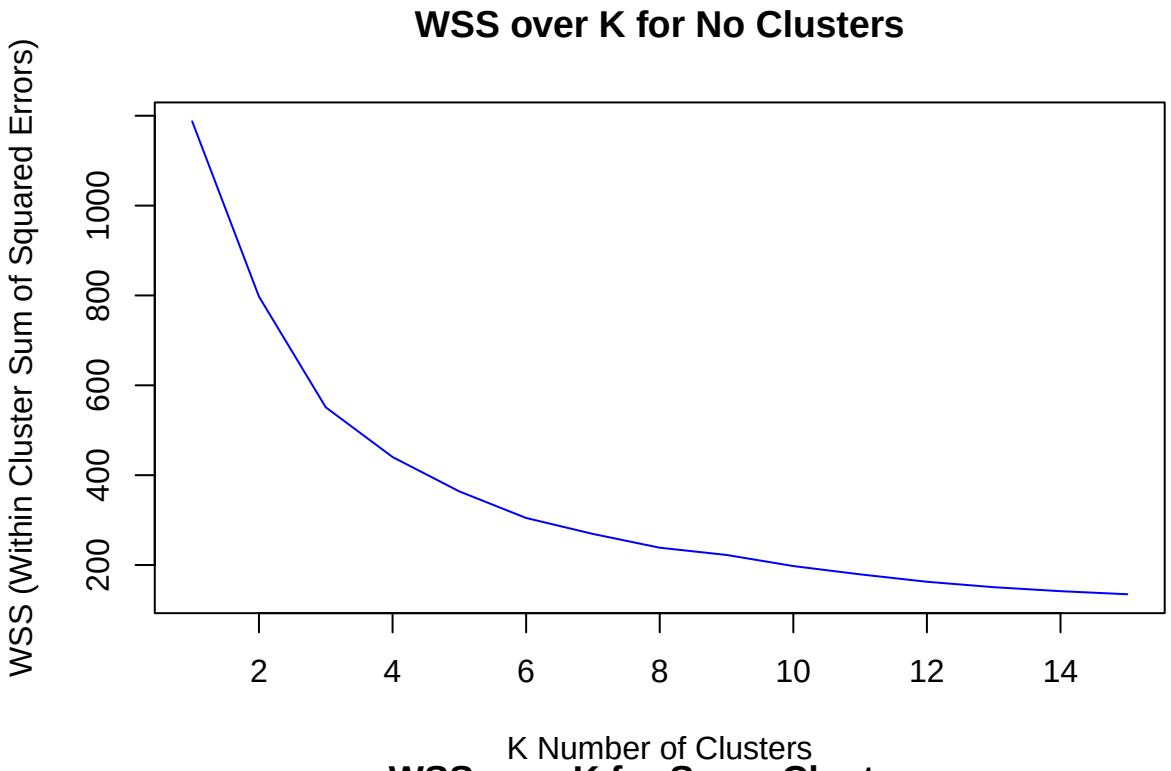
# Will store the WSS values for each k for each dataset
WSS_per_set <- list()
datasets <- list(nocluster_df, some_cluster_df, clear_cluster_df)
names(datasets) <- c("No Clusters", "Some Clusters", "Clear Clusters")

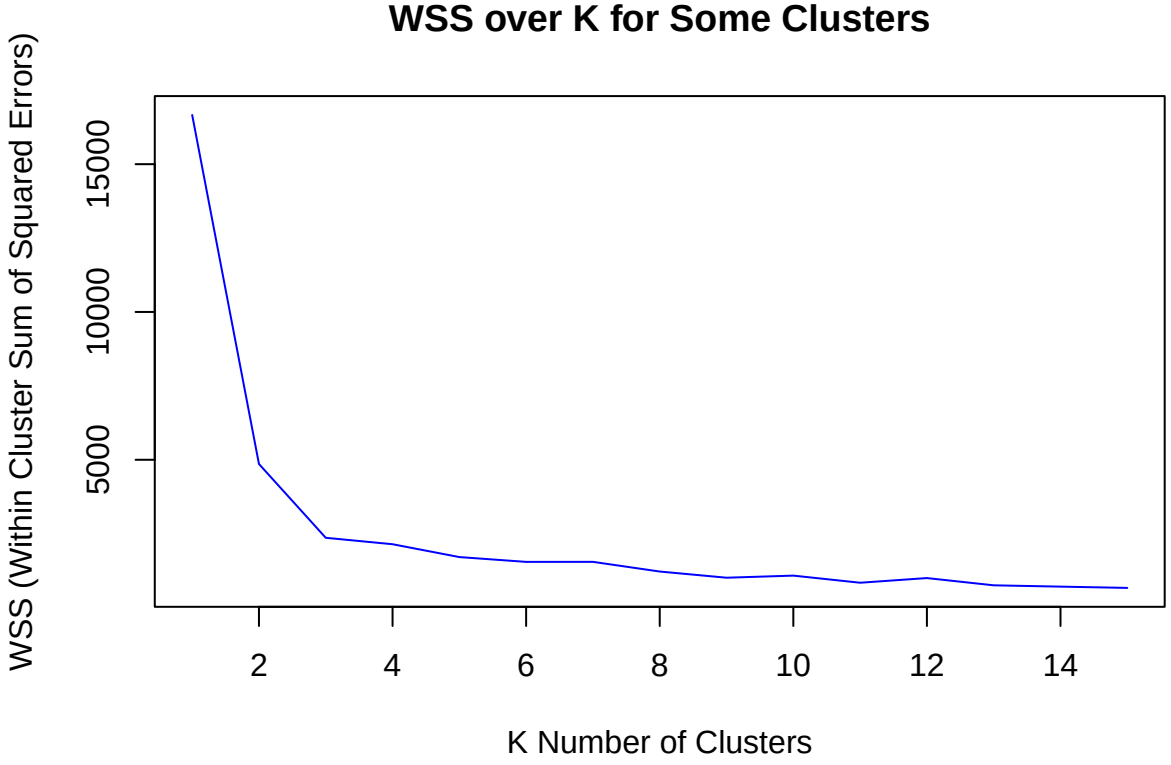

for (d in 1:3) {
  data <- datasets[[d]]
  WSS_values <- numeric(length(k_plotting))

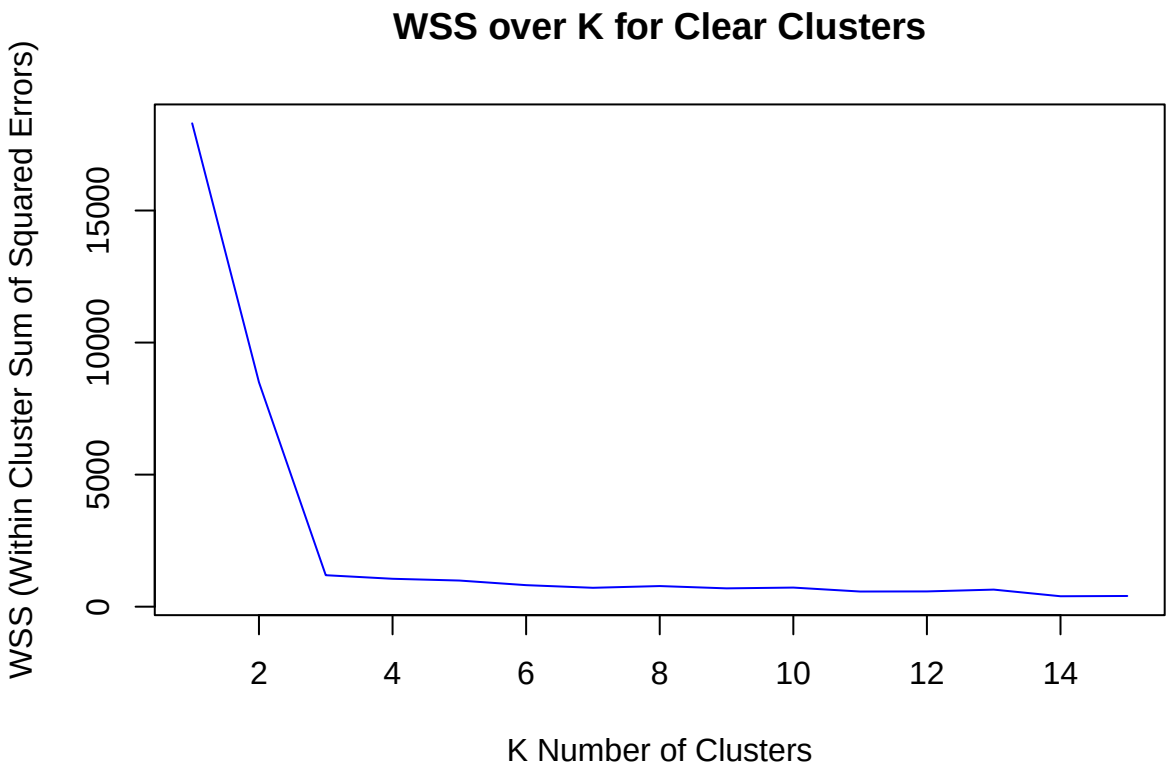
  # For each k call kmeans function to find WSS for that k for that dataset
  for (k in k_plotting) {
    result <- kmeans_algorithm(k, data)
    WSS_values[k] <- result$WSS
  }

  WSS_per_set[[d]] <- WSS_values

  # Plot for each dataset
  plot(x = k_plotting, y = WSS_values, type = "l", col = "blue",
       xlab = "K Number of Clusters", ylab = "WSS (Within Cluster Sum of Squared Errors)",
       main = paste("WSS over K for", names(datasets)[d]))
}
```


WSS over K for No Clusters
WSS (Within Cluster Sum of Squared Errors)
200
400
600
800
1000
2
4
6
8
10
12
14
K Number of Clusters


WSS over K for Some Clusters
WSS (Within Cluster Sum of Squared Errors)
5000
10000
15000
2
4
6
8
10
12
14
K Number of Clusters




```r
# PART THREE: Analyzing Stability Measure Across 10 Different Seeds

seeds <- c(125, 236, 745, 658, 397, 956, 421, 288, 194, 875)
stability_results <- list()

for (d in 1:3) {
  data <- datasets[[d]]
  n <- nrow(data)

  # Store cluster assignments for all seeds
  cluster_matrix <- matrix(0, nrow = n, ncol = length(seeds))

  for (s in 1:length(seeds)) {
    set.seed(seeds[s])
    result <- kmeans_algorithm(k = 3, data = data)
    cluster_matrix[, s] <- result$clustering
  }

  # Stability matrix
  stability_matrix <- matrix(0, nrow = n, ncol = n)

  for (i in 1:n) {
    for (j in 1:n) {
      stability_matrix[i, j] <- mean(cluster_matrix[i, ] == cluster_matrix[j, ])
    }
  }

  stability_results[[d]] <- stability_matrix
}
```

```r
mean(stability_results[[1]])
```

```
## [1] 0.3346189
```

```r
mean(stability_results[[2]])
```

```
## [1] 0.3334389
```

```r
mean(stability_results[[3]])
```

```
## [1] 0.35008
```

If we observe the means of the stability measures for each of the datasets, the dataset with clear clusters has the least stability, and the dataset with some clustering has the most stability, although the stability values are very similar for all the datasets.